

RepoZero: Can LLMs Generate a Code Repository from Scratch?

Zhaoxi Zhang*

Peking University
zhaoxizhang25@stu.pku.edu.cn

Yiming Xu

Peking University
teddyxu@stu.pku.edu.cn

Jiahui Liang

Baidu Inc
liangjiahui03@baidu.com

Weikang Li[†]

Baidu Inc
wavejkd@pku.edu.cn

Yunfang Wu[†]

Peking University
wuyf@pku.edu.cn

Abstract

Large Language Models (LLMs) have recently shown remarkable progress in code generation, yet their ability to construct complete software repositories from scratch remains poorly understood. A fundamental bottleneck is the lack of verifiable and scalable evaluation: existing benchmarks either focus on patch-based editing or rely on human or LLM-based judgments, which introduce bias and limit reproducibility. In this work, we present RepoZero, the first benchmark that enables fully automated, execution-based verification of repository-level generation from scratch. Our key idea is to reformulate generation as repository reproduction: given only API specifications, an agent must re-implement an entire repository such that its behavior matches the original implementation. This design allows for strict black-box validation via output equivalence, while naturally supporting large-scale construction by reusing existing open-source repositories. To further mitigate data leakage and shortcut solutions, we introduce cross-language constraints and a sandboxed evaluation protocol. Building on this benchmark, we propose an Agentic Code-Test Evolution (ACE) framework that performs iterative test generation and error-driven refinement, enabling effective test-time scaling for repository-level synthesis. Extensive experiments across multiple state-of-the-art LLMs and agent frameworks reveal that even the strongest LLM agents achieve only limited pass rates (30% - 55%), exposing a substantial gap between current capabilities and real-world software development requirements. Our results establish RepoZero as a challenging, scalable, and reliable testbed for end-to-end code generation, and highlight self-verification via test generation as a critical direction for advancing LLM-based coding agents. Code is available at <https://github.com/JesseZZZZZ/RepoZero>.

1 Introduction

Recent advances in LLM-based coding have drawn increasing attention toward fully autonomous end-to-end software development. State-of-the-art large language models [Liu et al., 2025a, Yang et al., 2025, Team et al., 2026, Xiao et al., 2026, Zeng et al., 2026] have been integrated into coding agents [Wang et al., 2024, Gao et al., 2025, Zhang et al., 2025] capable of completing complex programming tasks with minimal or no human intervention. From the perspective of benchmark difficulty, coding tasks can generally be categorized along two orthogonal dimensions. The first dimension concerns the scope of code modification, including: (1) single-file coding and (2) repository-level coding. The

*Work done during internship at Baidu Inc.

[†]Correspondence Author

Table 1: Comparison with existing coding benchmarks on multiple dimensions. **Repository** (generating a repository rather than a single code snippet), **Multi-lang.** (multiple programming languages), **From-scratch Gen.** (generation with no pre-given code), **Execution** (verifying the executed results), **Test Suites** (test cases rather than LLM-as-judge), **Scalability** (automatic pipeline rather than human-required), and **Leakage Defense** (preventing data leakage).

Benchmark	Repository	Multi-lang.	From-scratch Gen.	Execution	Test Suites	Scalability	Leakage Defense
HumanEval [Chen et al., 2021]	✗	✗	✗	✓	✓	✗	✓
SWE-bench [Jimenez et al., 2024]	✓	✗	✗	✗	✓	✗	✗
Multi-SWE-bench [Zan et al., 2025a]	✓	✓	✗	✗	✓	✗	✗
FEA-bench [Li et al., 2025]	✓	✗	✗	✗	✓	✗	✗
EvoCodeBench [Zhang et al., 2026]	✓	✗	✓	✗	✓	✗	✗
CodeS [Zan et al., 2025b]	✓	✗	✓	✗	✗	✗	✗
E2EDevBench [Liu et al., 2025b]	✓	✗	✓	✗	✗	✗	✓
NL2Repo [Ding et al., 2025]	✓	✗	✓	✓	✓	✗	✗
RepoZero (Ours)	✓	✓	✓	✓	✓	✓	✓

second dimension concerns the nature of the programming task itself, including: (1) debugging, (2) incremental development, and (3) generation from scratch.

Current coding agents have largely mastered single-file editing. However, evaluating repository-level modifications remains a burgeoning frontier. Benchmarks such as SWE-bench [Jimenez et al., 2024], Multi-SWE-bench [Zan et al., 2025a], and FEA-bench [Li et al., 2025] have pioneered the assessment of repo-level edits and incremental development in real-world scenarios. While these benchmarks leverage patch-based verification and unit testing, the domain of from-scratch repository generation remains relatively unexplored. A primary obstacle is the scarcity of pre-existing unit tests available online for newly generated projects. Consequently, current benchmarks for from-scratch generation, such as CodeS [Zan et al., 2025b], EvoCodeBench [Zhang et al., 2026], and NL2RepoBench [Ding et al., 2026], predominantly rely on human-in-the-loop or LLM-as-a-judge frameworks. These methods, however, introduce subjective biases from both human evaluators and language models, leading to potentially unreliable evaluation outcomes.

Data leakage presents another formidable challenge for repository-level coding benchmarks. Since modern large language models (LLMs) are extensively pre-trained on massive GitHub corpora [Zeng et al., 2026, Xiao et al., 2026], it is increasingly difficult to discern whether their performance stems from genuine reasoning capabilities or mere rote memorization. While some studies [Zhang et al., 2026] attempt to mitigate this by utilizing rare or recently published repositories, this approach inherently constrains the dataset scale, thereby precluding large-scale evaluation. Furthermore, such "clean" datasets are subject to rapid contamination post-publication, necessitating frequent and labor-intensive updates to maintain their integrity.

To address these challenges, we propose **RepoZero**, the first verifiable benchmark specifically designed to evaluate the from-scratch repository-level generation capabilities of LLM agents. RepoZero achieves rigorous verification by reformulating repository-level generation as a repository reproduction task. Specifically, for a target repository (e.g., a pip-installed Python package), we first employ an LLM to generate test files that invoke the repository’s APIs; for each test file, the LLM generates several test cases, which are then filtered to retain only successful test cases as ground truth. An agent is then tasked with generating an entire target repository that replicates the source repository’s functionality and API signatures. Evaluation is performed by executing the generated target repository against the saved test cases and comparing the outputs with those of the source repository. To mitigate data leakage, we mandate cross-language synthesis—requiring agents to implement the repository in a different programming language—and prohibit the use of external site packages. Furthermore, we introduce an Agentic Code-Test Evolution (ACE) workflow for test-time scaling. Unlike existing frameworks, where the lack of a verifiable environment makes retrieving ground-truth outputs for LLM-generated tests impossible, RepoZero leverages the source repository as an oracle, providing a deterministic and automated gold standard for evaluation.

In summary, our primary contributions are as follows:

1. **Introduction of RepoZero:** We establish the first verifiable and highly scalable benchmark tailored for repository generation from scratch, bridging the evaluation gap in existing repo-level coding tasks.
2. **Test-Time Scaling Framework:** We propose a novel agentic framework centered on test-time scaling, which significantly enhances the success rate of complex, repository-level synthesis.

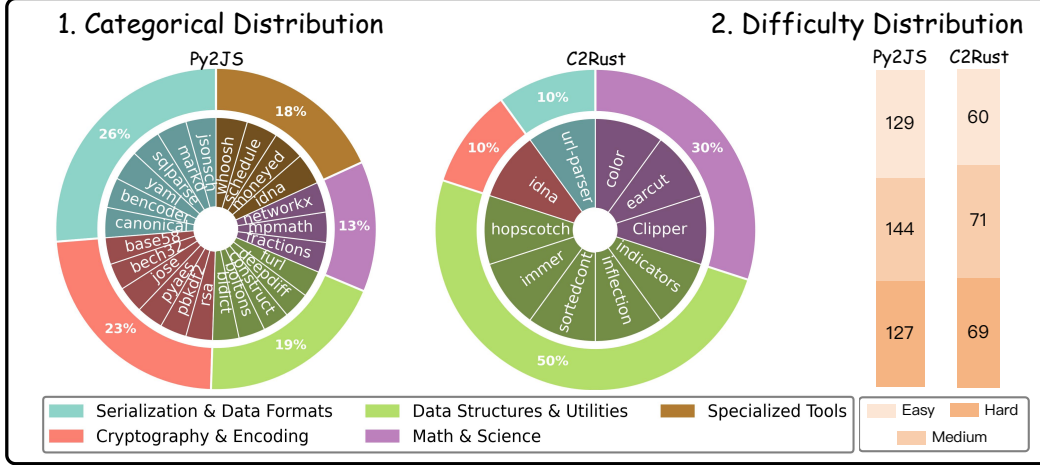


Figure 1: Demonstration of (1) categorical distribution, and (2) difficulty distribution of RepoZero-Py2JS (left) and RepoZero-C2Rust (right).

3. Comprehensive Benchmarking: We conduct an extensive evaluation of state-of-the-art models and agentic scaffolds, providing critical insights and a robust foundation for future research in autonomous software engineering.

2 Benchmark: RepoZero

2.1 Statistics of RepoZero

RepoZero consists of two subsets: **RepoZero-Py2JS**, where the source repositories are implemented in Python, and the agent is required to reimplement them in JavaScript, and **RepoZero-C2Rust**, where the source repositories are written in C/C++ and the agent is tasked with reimplementing them in Rust. As illustrated in Fig. 1, RepoZero covers a diverse range of repository categories. All repositories are manually curated and further preprocessed following the workflow described in Sec. 3. We employ LLMs to estimate the difficulty of each sample under several rubrics: Lines of Code (LOC), API counts, and input/output complexity. We apply majority voting (Claude-4.6-Opus, GLM-5, and DeepSeek-V3.2) to obtain the final estimation. The datasets are accordingly partitioned into *Easy*, *Medium*, and *Hard* subsets. From the curated repositories, we construct 400 samples for RepoZero-Py2JS and 200 samples for RepoZero-C2Rust. The source repositories of these samples are manually selected. Because the remaining pipeline is automated after repository selection, the benchmark can be continuously updated to mitigate potential data leakage (once released, RepoZero will inevitably be collected as part of the training data of foundational models).

2.2 Task Definition and Challenges

In a repository reproduction task, an LLM agent is provided with: (1) the functional specifications of the source repository’s APIs, (2) four white-box test cases (comprising inputs, ground-truth outputs, and descriptions), and (3) constraints to preclude "shortcut" behaviors, such as cross-language embedding or direct API delegation. The target repository is compiled, where applicable, and evaluated via black-box testing to ensure its outputs strictly align with the source repository’s baseline. This task presents four primary challenges for LLM agents: (1) Functional Synthesis, translating high-level API specifications into concrete implementations; (2) Modular Reasoning, resolving inter-module dependencies across multiple files; (3) Iterative Self-Correction, refining code through autonomous test design and execution; and (4) Long-context Retention, maintaining architectural coherence over extensive reasoning horizons. As shown in Sec. 6, current agents exhibit several limitations in navigating these complexities.

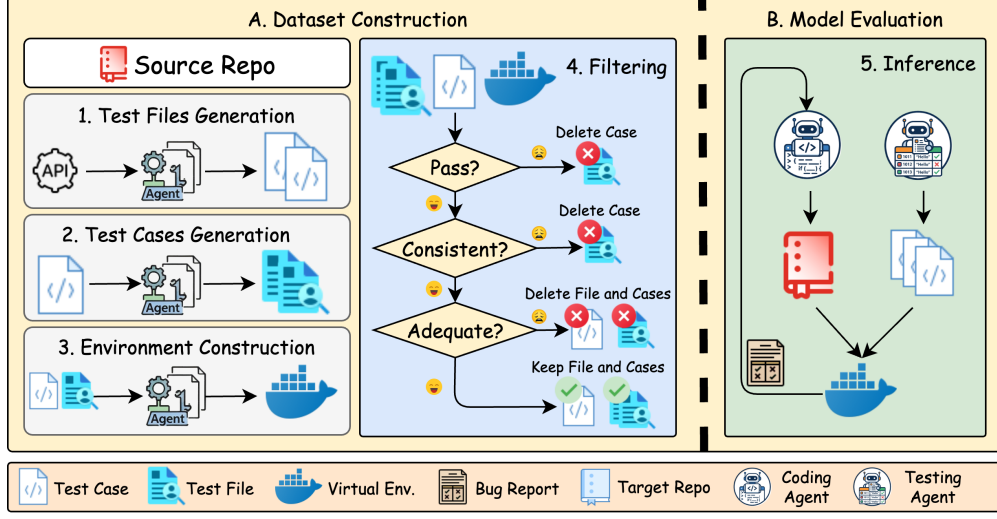


Figure 2: Illustration of (A) the construction of the RepoZero benchmark, and (B) agentic behavior during the evaluation stage. The loop ends when all cases produced by the testing agent succeed.

2.3 Evaluation Pipeline

We employ two popular coding agents: OpenHands-bash [Wang et al., 2024, Sutawika et al., 2026] and Mini-SWE-Agent [Yang et al., 2024a]. These agents can interact with the computer system via a series of tools, for instance, a terminal, where the agent can run commands. The agent is prompted with the task information and prompted to generate a repository with an assigned entry point. After finishing the generation, black-box tests are applied to test the quality of the target repository.

To ensure a rigorous evaluation, we address the potential for "cheating" behaviors—such as accessing ground-truth repositories or manipulating test suites—frequently observed in recent LLM agent benchmarks [Jimenez et al., 2024, Merrill et al., 2026]. We implement a multi-layered protocol to maintain the integrity of the evaluation process:

Black-box Testing To mitigate the risk of agents overfitting to specific test outcomes, we implement a black-box evaluation framework. Only four representative white-box test cases are included in the prompt for contextual guidance, while the remainder of the evaluation suite is withheld. By prioritizing semantic correctness over superficial execution results, this methodology ensures that agents cannot bypass rigorous verification through trivial output matching or heuristic shortcuts.

Environment Isolation and Security Evaluation is performed within isolated Docker containers characterized by restricted file-system permissions. To maintain the integrity of the benchmark, the source repository’s test files are configured as read-only, and the hidden evaluation suite is never exposed to the agent’s accessible file system. These safeguards prevent agents from manipulating or circumventing challenging test cases to artificially inflate performance metrics.

Execution Constraints and Cross-Language Integrity We impose stringent limitations on system-level commands to preclude non-generalizable solutions. Specifically, agents are prohibited from importing external packages into the target repository. Furthermore, we enforce strict language consistency: for instance, if an agent is tasked with porting a Python library (e.g., mpmath) to JavaScript, the use of bridge commands or cross-language invocations (e.g., executing Python scripts via shell wrappers) is strictly forbidden. Such constraints necessitate that agents rely on complex logical reasoning and cross-language synthesis rather than delegating tasks to external APIs.

3 Benchmark Construction

In this section, we illustrate the dataset construction process of our benchmark. First, we manually select several open-source repositories from GitHub, and we prompt an LLM agent to generate test

files (each successful file stands for one sample in the dataset) that invoke APIs from the repository. Second, we prompt a test-case generator to generate test cases for each file. Third, we conduct filtering techniques for all test cases and test files.

3.1 Sample Generation

We manually curate a set of source repositories, denoted as R_o , and leverage an LLM to generate corresponding test files. To modulate task difficulty, we employ strict prompting to regulate the number of API calls per file, ranging from 1 to 20. We emphasize that repository selection is governed by human expert judgment based on three criteria. **Determinism**: The repository must be deterministic to ensure consistent string-level comparisons. **Open-source Integrity**: The repository must be fully open-source, excluding non-public components such as proprietary built-in hash tables or frozen model weights. **Architectural Complexity**: The repository must be sufficiently complex to preclude single-file implementations, thereby rigorously evaluating repository-level coding capabilities.

3.2 Test Case Generation

We employ an LLM as a generator to produce evaluation datasets. For each target repository, the LLM is provided with the repository name and the corresponding test files to extract the underlying API specifications. Based on these contexts, the LLM is prompted to generate a comprehensive suite of input-output pairs. However, since the LLM has limited visibility into the entire codebase and the source repositories themselves may contain inconsistencies, the initial outputs are often unpredictable. Consequently, a rigorous filtering pipeline is essential to ensure the quality and reliability of the generated test cases.

3.3 Environment Configuration

To facilitate execution, we instantiate an isolated Docker container for each sample, equipped with standard development tools (e.g., Python, g++, etc.). We leverage OpenHands [Wang et al., 2024] to dynamically resolve and install the dependencies required by the source test files. An environment is validated as "successful" only if it can execute the generated test files and pass at least five baseline cases without runtime errors. This stage ensures that the environment is correctly configured to support the subsequent large-scale verification of the entire test suite.

3.4 Data Filtering and Ground-Truth Verification

With the repositories, test files, and candidate test cases assembled, we perform a multi-stage filtering process to curate valid evaluation samples. Each test case is executed 20 times within its designated environment to ensure stability. Test cases are discarded if they trigger: (1) runtime exceptions, or (2) non-deterministic outputs. For example, outputs involving memory addresses (pointers) or time-dependent variables are eliminated due to their inherent volatility. This rigorous pruning ensures that each remaining test case is both executable and deterministic, with its stable output serving as the definitive ground-truth for model evaluation. Samples are discarded if the number of valid test cases is fewer than 10.

4 Agentic Code-Test Evolution Workflow

Recent studies have widely acknowledged that jointly generating test cases alongside code can substantially improve code generation performance [Ding et al., 2025]. However, the code-test feedback loop has not been broadly adopted in existing workflows for two primary reasons. First, LLMs do not consistently exhibit the tendency to generate test cases without explicit prompting or a predefined procedural framework. Second, test cases generated by LLMs are typically not independently verifiable; consequently, such cases can only assess code executability rather than functional correctness.

Inspired by Agentless [Xia et al., 2024], which replaces fully autonomous agentic procedures with a structured workflow to enhance LLM-based bug localization, we propose the **Agentic Code-Test**

Table 2: Evaluation with 🙌 Openhands-bash on RepoZero. We present the pass rate of models across Easy, Medium, and Hard task difficulties.

Model	🔗 Py2JS 📄				🔗 C2Rust 📄			
	Easy	Medium	Hard	Avg.	Easy	Medium	Hard	Avg.
🔲 GLM-5	37.21	23.61	22.05	27.46±2.29	31.67	20.00	20.29	23.47±2.34
🔲 GLM-5.1	46.51	38.89	16.54	34.19±2.38	45.90	31.43	28.99	35.05±2.19
🔲 Kimi-K2.5	36.36	33.02	24.74	31.30±2.97	40.98	30.00	28.99	33.00±1.97
🔲 Kimi-K2.6	47.29	27.78	25.20	33.25±2.36	50.82	30.00	36.23	38.51±2.03
🔵 Ernie-5.0	27.91	18.75	11.81	19.46±2.02	22.95	12.86	4.35	13.97±2.36
🔴 Minimax-M2.5	29.97	22.23	18.89	22.72±1.92	36.07	18.57	31.88	28.48±2.31
🔴 Minimax-M2.7	34.88	29.17	16.54	27.15±2.15	37.70	25.71	33.33	32.12±2.91
🔵 DeepSeek V3.1	34.11	25.69	18.11	26.08±2.26	43.33	30.00	26.09	32.49±2.00
🔵 DeepSeek V3.2	37.21	31.25	18.11	29.01±2.34	48.33	28.57	26.09	33.55±2.52
🔵 DeepSeek V4 Flash	38.76	32.64	20.47	30.74±2.33	40.98	34.92	31.88	35.41±2.38
🔵 DeepSeek V4 Pro	48.84	38.19	32.28	39.78±2.51	40.98	32.86	33.33	35.53±2.40
🌟 Claude-4.6-Sonnet	59.69	49.31	45.67	51.46±2.53	53.33	45.71	36.23	44.60±2.74

Evolution (ACE) workflow, a dedicated framework designed for RepoZero. As illustrated in Fig. 2 (B), ACE adopts an iterative code–test feedback loop.

RepoZero provides a suitable testbed for evaluating ACE. This is primarily because the ground-truth outputs for RepoZero samples are obtained by executing the source repository itself, which enables the generation of an effectively unlimited number of test cases while ensuring the correctness and reliability of their corresponding labels.

In practice, ACE serves as a general framework that can be integrated with arbitrary coding agents. The workflow begins with a coding agent that generates a target repository from scratch based on the APIs of the source repository. Subsequently, a testing agent produces multiple test cases and executes the source repository to obtain the corresponding ground-truth outputs. Notably, the filtering strategy described in Sec. 3.2 is also applied at this stage. If the generated repository successfully passes all test cases, the iterative loop terminates. Otherwise, the resulting error messages are fed back to the coding agent, which then revises the generated repository accordingly.

5 Experimental Setup

Models and Scaffolds We evaluate our benchmark using two widely recognized agent scaffolds for software engineering: OpenHands-bash [Sutawika et al., 2026] and Mini-SWE-Agent [Yang et al., 2024a]. These frameworks are integrated with state-of-the-art LLMs, including Kimi-K2.5, Kimi-K2.6 [MoonShot, 2026], GLM-5, GLM-5.1 [Zeng et al., 2026], DeepSeek-V3.1, DeepSeek-V3.2 [DeepSeek-AI, 2025], DeepSeek-V4 [DeepSeek-AI, 2026], Ernie-5.0 [Wang et al., 2026], Minimax-M2.5, Minimax-M2.7 [MinimaxAI, 2026] and Claude-4.6 [Anthropic, 2026]. To ensure maximum flexibility, we do not impose an explicit cap on the number of tool-calling iterations; the only constraint is the maximum output token limit inherent to the base models. Furthermore, agents are permitted full access to context management techniques and retrieval-augmented tools to facilitate complex problem-solving.

Metrics We adopt the Pass Rate (PR) as our primary evaluation metric. A sample is considered successful ($PR = 1$) if and only if the outputs from both the source and target repositories exhibit strict string-level consistency; otherwise, it is assigned a value of 0. Additionally, we report the Success Rate (SR) across test cases. While the SR (see Appendix A) offers a complementary perspective, the PR is utilized as the more robust and definitive measure of functional equivalence. Results are reported as mean $\pm$ bootstrap standard deviation.

Table 3: Evaluation with  Mini-SWE-Agent on RepoZero. We present the pass rate of models across Easy, Medium, and Hard task difficulties.

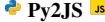
Model	 Py2JS 				 C2Rust 			
	Easy	Medium	Hard	Avg.	Easy	Medium	Hard	Avg.
 GLM-5	55.81	43.75	37.01	45.52±2.48	49.18	37.14	30.43	38.51±1.98
 Kimi-K2.5	44.19	32.64	24.41	33.76±2.48	54.10	40.00	37.68	43.53±2.33
 Ernie-5.0	34.11	15.28	13.39	20.77±2.08	26.23	14.29	17.39	19.97±2.04
 Minimax-M2.5	49.91	29.25	22.68	30.63±2.72	32.79	24.29	20.29	25.39±2.31
 DeepSeek V3.1	58.91	45.14	26.77	43.76±2.50	50.82	37.14	31.88	39.45±2.51
 DeepSeek V3.2	52.71	44.44	35.43	44.20±2.59	49.18	38.57	26.09	37.41±2.78
 Claude-4.6-Sonnet	65.12	54.17	44.88	54.70±2.55	59.02	45.71	39.73	47.58±2.90

Table 4: Evaluation on  Py2JS  with DeepSeek V3.1 as the backbone model. We present the pass rate of models across Easy, Medium, and Hard task difficulties, and the maximum retry times are set to 0 (coding only), 1 (coding-testing-refining), and 2 (coding-testing-refining-testing-refining).

Method	 Openhands-bash				 Mini-SWE-Agent			
	Easy	Medium	Hard	Avg.	Easy	Medium	Hard	Avg.
 Retry-0	34.11	25.69	18.11	26.08±2.26	58.91	45.14	26.77	43.76±2.50
 Retry-1	45.74	37.50	25.20	36.15±2.38	58.91	47.92	29.92	45.67±2.44
 Retry-2	51.14	41.51	37.11	42.88±2.93	64.34	51.39	40.16	52.06±2.38

6 Results and Analysis

6.1 Main Evaluation

The primary experimental results for the RepoZero-Py2JS and RepoZero-C2Rust benchmarks are presented in Table 2 and Table 3, while results in Table 5 illustrate the performance across multiple categories. Overall, all evaluated models exhibit suboptimal performance on these benchmarks regardless of whether the OpenHands-bash or Mini-SWE-Agent scaffold is employed. Notably, Claude-4.6-Sonnet consistently achieves the highest performance across all evaluation metrics.

Our analysis reveals that Mini-SWE-Agent generally outperforms OpenHands-bash. This performance gap can likely be attributed to more sophisticated context engineering in Mini-SWE-Agent, whereas OpenHands-bash represents a baseline agent configuration restricted to a rudimentary bash interface. Furthermore, as illustrated in Fig. 4 (right), OpenHands-bash fails even on test cases explicitly provided within the initial prompt. This observation suggests that without robust, carefully designed context management, agents struggle to retain and leverage critical information during complex, long-context reasoning tasks.

Benchmark Validity While the semi-synthetic nature of RepoZero precludes inherent quality guarantees, its empirical validity is substantiated by consistent performance alignment within established model families. Adhering to the *Benchmark*² principle [Qian et al., 2026], our results (Tables 2 and 3) demonstrate that successive iterations of the Kimi, GLM, DeepSeek, and Minimax series exhibit progressive performance gains. Furthermore, the performance hierarchy—especially Claude and Ernie—aligns with broader industry benchmarks. This structural consistency across diverse model tiers underscores the reliability of RepoZero.

Cost-Performance Trade-offs Figure 3 illustrates the general correlation between inference expenditure and model efficacy, where the pass rate typically scales with per-sample cost. While Claude-4.6-Sonnet defines the upper bound for both metrics, this relationship is not strictly monotonic. Notably, GLM-5.1 significantly outperforms GLM-5 despite a nearly identical cost structure, highlighting how architectural advancements can decouple performance from computational overhead. However, the persistent performance gap between these leading open-source models and SOTA closed-source systems (e.g., Claude) remains a primary challenge for open-source development.

6.2 Test-time Scaling via ACE

Table 4 summarizes the ACE (a case study is provided in Fig. 4 left) performance of OpenHands-bash and Mini-SWE-Agent on the RepoZero-Py2JS benchmark, both of which are underpinned by the DeepSeek V3.1 backbone. The empirical results demonstrate a substantial performance gain when test cases are dynamically generated and executed during the inference phase. These findings from the ACE framework underscore two pivotal directions for the evolution of repository-level coding agents: (1) generating high-quality, executable, and verifiable test cases; and (2) integrating predefined test cases—whether human-authored or LLM-generated—in automated software engineering workflows.

6.3 Failure Analysis

We categorize the identified failure modes into four distinct groups: (1) failure to generate the executable code; (2) failure to pass white-box test cases (which are provided to the agent as illustrative examples); (3) general runtime errors, such as arithmetic overflow or dependency conflicts; and (4) output mismatches between the source and target repositories during black-box testing.

The performance breakdown of open-source models with the OpenHands-bash scaffold is illustrated in Fig. 4 (right). Our analysis reveals that runtime errors occur infrequently, whereas other failure modes predominate. Based on these observations, we offer the following insights for the design of future autonomous coding agents:

Long-term Memory and Context Retention A non-negligible portion of failures was observed even within white-box test cases, a phenomenon primarily attributable to deficiencies in the agents’ long-context management. Despite the inclusion of all white-box constraints in the initial prompt, agents frequently exhibit "contextual drift" during extended reasoning sequences, losing track of critical requirements as the trajectory length increases.

Autonomous Environment Utilization We observe that most high-performing agents proactively design and execute automated unit tests within the provided environment. This self-correcting capability is instrumental in mitigating runtime errors, which are infrequent across our evaluation suite. Integrated with the empirical data in Table 4, these findings underscore that the ability to leverage an executable environment for test-driven development is a fundamental determinant of success in complex software engineering tasks.

Discrepancy Between Runnability and Correctness While many existing benchmarks evaluate repository-level performance based solely on execution success, our results expose a substantial gap between runnability and semantic correctness. Notably, approximately 40% of the executable code generated by agents fails to match the deterministic output of the source repository. This discrepancy reinforces the necessity of the more stringent, output-verified evaluation framework of RepoZero.

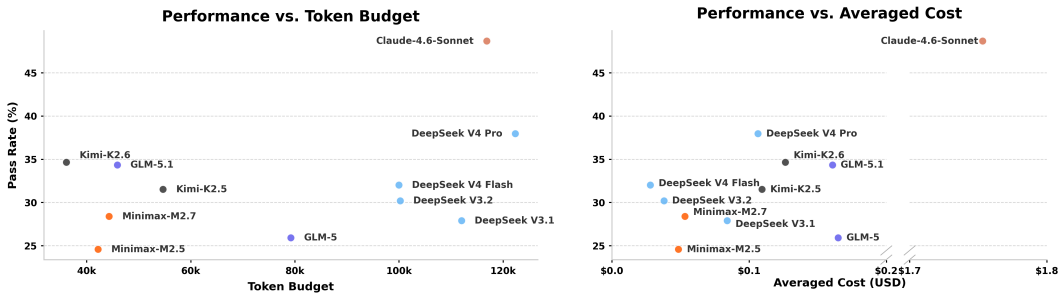


Figure 3: **Left:** pass rate vs. average token budget. **Right:** pass rate vs. average cost in USD. All models are evaluated with 🤖 OpenHands-bash.

7 Related Works

7.1 Repo-level Coding Benchmarks

Traditional coding benchmarks [Chen et al., 2021, Lu et al., 2021, Gu et al., 2024] primarily assess the capability of LLMs to generate isolated code snippets, a scope that fails to capture the complexities of real-world software engineering. To bridge this gap, SWE-bench [Jimenez et al., 2024] introduced the first evaluation framework for repository-level bug fixing. This line of research has been further extended by Multi-SWE-bench [Zan et al., 2025a], SWE-bench-Multimodal [Yang et al., 2024b], and SWE-bench-Pro [Deng et al., 2025], which incorporate multiple modalities and diverse programming languages. Complementing the focus on maintenance, FEA-bench [Li et al., 2025] emerged as the first benchmark to evaluate LLMs’ ability to implement new features within existing repositories. More recently, the challenge of from-scratch repository generation has gained significant attention. Although CodeS [Zan et al., 2025b] and Commit0 [Zhao et al., 2024] automate repository collection from GitHub, these approaches struggle with scalability and are inherently susceptible to data leakage. While NL2Repo [Ding et al., 2025] and EvoCodeBench [Zhang et al., 2026] provide manually curated natural-language descriptions and test suites for Python repositories, and E2EDevBench [Liu et al., 2025b] utilizes GitHub-sourced repositories with "LLM-as-a-judge" rubrics, a fundamental bottleneck remains: the difficulty of constructing comprehensive test cases for repository-level generation without intensive human labor. To address this, we present a novel benchmark that evaluates an LLM agent’s ability to generate a repository from scratch by re-implementing it based on the source APIs. Our framework represents the first benchmark that facilitates both automated test-case verification and large-scale production, ensuring both evaluation and scalability.

7.2 LLM Coding Agents

Following the reasoning-action loop [Yao et al., 2022], SWE-Agent [Yang et al., 2024a], and OpenHands [Wang et al., 2024] are the first LLM agents to conduct coding operations within a repository. LocAgent [Chen et al., 2025], CoSIL [Jiang et al., 2025], and GraphLocator [Liu et al., 2025c] build a call graph for the repository and implement graph operations to assist repo-level reasoning. RepoSearcher [Ma et al., 2025], RepoNavigator [Zhang et al., 2025], and CodeScout [Sutawika et al., 2026] apply reinforcement learning to train the agent for bug fixing. The aforementioned agents are mainly designed for bug fixing, which is verifiable via unit tests. For code generation from scratch, as far as we know, there are only a limited number of agents that are specialized for the task. CodeS [Zan et al., 2025b] performs end-to-end repository generation by first designing the sketch of the repository, then filling in the code. The difficulty for rule-based verification is one of the most critical reasons that hinder the development of agents that generate repositories from scratch.

8 Conclusion

This paper presents RepoZero, the first scalable and verifiable benchmark for end-to-end repository generation. Unlike traditional code-completion tasks, RepoZero requires models to construct entire software projects from scratch. To ensure evaluation integrity and mitigate data contamination, we introduce a cross-language protocol across two primary subsets, *C2Rust* and *Py2JS*, encompassing 600 test files.

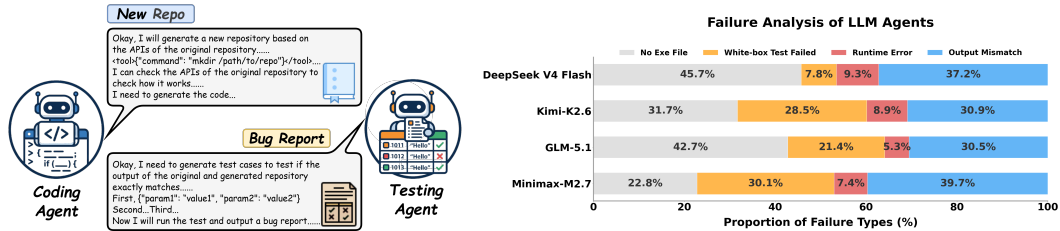


Figure 4: **Left:** an example illustrating the agentic code-test evolution framework. **Right:** analysis of failure conditions of RepoZero.

Our experimental analysis reveals that even state-of-the-art models, supported by advanced scaffolds, achieve only moderate success (approximately 40%), and self-verification is an important technique. These findings underscore that while the iterative "code-test" loop is a promising paradigm, a significant gap remains in agents' self-verification capabilities. Future progress in repository-level generation will necessitate a more robust integration of autonomous, high-quality test suite synthesis.

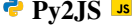
Limitations and Future Work We propose three primary directions to advance this field: (1) developing specialized training paradigms, such as reinforcement learning or distillation, to enhance global repository-level reasoning; (2) devising metrics and constraints to improve the architectural readability and structural integrity of generated codebases; and (3) expanding the benchmark to include large-scale, multilingual, and industry-level software development scenarios.

References

- Anthropic. Introducing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, February 2026. Accessed: 2026-04-23.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided llm agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, 2025.
- DeepSeek-AI. Deepseek-v3.1 release. <https://api-docs.deepseek.com/news/news250821>, August 2025. Accessed: 2026-04-29.
- DeepSeek-AI. Introducing deepseek v4. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>, February 2026. Accessed: 2026-04-23.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, et al. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025.
- Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents, 2026. URL <https://arxiv.org/abs/2512.12730>.
- Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchun Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.
- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.
- Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Cosil: Software issue localization via llm-driven code repository graph searching. *arXiv e-prints*, pages arXiv–2503, 2025.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. FEA-bench: A benchmark for evaluating repository-level code generation for feature implementation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17160–17176, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.839. URL <https://aclanthology.org/2025.acl-long.839/>.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025a.
- Jingyao Liu, Chen Huang, Zhizhao Guan, Wenqiang Lei, and Yang Deng. E2edev: Benchmarking large language models in end-to-end software development task. *arXiv preprint arXiv:2510.14509*, 2025b.
- Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. Graphlocator: Graph-guided causal reasoning for issue localization. *arXiv preprint arXiv:2512.22469*, 2025c.
- Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*, 2021.
- Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. Tool-integrated reinforcement learning for repo deep search. *arXiv preprint arXiv:2508.03012*, 2025.
- Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- MinimaxAI. Introducing minimax. <https://huggingface.co/MiniMaxAI/MiniMax-M2.7>, February 2026. Accessed: 2026-04-23.
- MoonShot. Introducing kimi. <https://huggingface.co/moonshotai/Kimi-K2.6>, February 2026. Accessed: 2026-04-23.
- Qi Qian, Chengsong Huang, Jingwen Xu, Changze Lv, Muling Wu, Wenhao Liu, Xiaohua Wang, Zhenghua Wang, Zisu Huang, Muzhao Tian, et al. Benchmark²: Systematic evaluation of llm benchmarks. *arXiv preprint arXiv:2601.03986*, 2026.
- Lintang Sutawika, Aditya Bharat Soni, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, Graham Neubig, et al. Codescout: An effective recipe for reinforcement learning of code search agents. *arXiv preprint arXiv:2603.17829*, 2026.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- Haifeng Wang, Hua Wu, Tian Wu, Yu Sun, Jing Liu, Dianhai Yu, Yanjun Ma, Jingzhou He, Zhongjun He, Dou Hong, et al. Ernie 5.0 technical report. *arXiv preprint arXiv:2602.04705*, 2026.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

- Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025a. URL <https://arxiv.org/abs/2504.02605>.
- Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Yafen Yao, Wei Li, Xiaolin Chen, Yongshun Gong, Bei Guan, Zhiguang Yang, Yongji Wang, Lizhen Cui, and Qianxiang Wang. Codes: Natural language to code repository via multi-layer sketch. *ACM Trans. Softw. Eng. Methodol.*, September 2025b. ISSN 1049-331X. doi: 10.1145/3768577. URL <https://doi.org/10.1145/3768577>. Just Accepted.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- Wentao Zhang, Jianfeng Wang, Liheng Liang, Yilei Zhao, HaiBin Wen, and Zhe Zhao. Evocodebench: A human-performance benchmark for self-evolving llm-driven coding systems, 2026. URL <https://arxiv.org/abs/2602.10171>.
- Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Zhixiang Wang, Kun Liang, Yang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, et al. One tool is enough: Reinforcement learning for repository-level llm agents. *arXiv preprint arXiv:2512.20957*, 2025.
- Wenting Zhao, Nan Jiang, Celine Lee, Justin T Chiu, Claire Cardie, Matthias Gallé, and Alexander M Rush. Commit0: Library generation from scratch. *arXiv preprint arXiv:2412.01769*, 2024.

Table 5: Evaluation on  **Py2JS** . We present the pass rate of models over 5 categories (see Fig. 1 for the definition of categories).

Method	 Openhands-bash					 Mini-SWE-Agent				
	Form	Crypto	Struc	Math/Sci	Spec	Form	Crypto	Struc	Math/Sci	Spec
 GLM-5	58.09	76.82	55.96	52.65	38.33	49.42	71.22	87.67	63.36	68.92
 Kimi-K2.5	56.17	59.91	85.75	56.14	39.94	45.26	58.19	70.25	50.25	62.16
 DeepSeek V3.1	35.08	44.42	63.88	44.42	46.40	54.08	71.40	84.96	65.26	54.57
 DeepSeek V3.2	48.08	49.11	70.84	42.07	55.92	57.17	68.92	90.51	61.99	67.72
 Ernie-5.0	47.24	33.06	37.48	42.60	35.11	14.87	40.27	48.62	42.56	54.66
 Minimax-M2.5	43.30	30.24	35.03	39.42	42.36	38.82	57.85	79.22	64.88	39.70
 Claude-4.6-Sonnet	42.92	66.26	78.89	67.14	68.77	47.10	73.92	88.42	71.02	66.79

Table 6: Evaluation on the full dataset. We present the averaged success rate across all test cases.

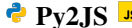
Model	 Py2JS 					 C2Rust 			
	Easy	Medium	Hard	Avg.		Easy	Medium	Hard	Avg.
 GLM-5	68.65	51.75	53.23	57.67		76.26	65.56	60.08	67.27
 Kimi-K2.5	61.21	56.08	59.83	58.88		67.36	53.98	49.44	56.85
 Ernie-5.0	50.76	34.96	32.33	39.22		49.07	32.25	34.01	38.23
 Minimax-M2.5	45.17	40.04	28.38	37.99		62.77	50.80	47.95	53.47
 DeepSeek V3.1	55.93	45.51	36.79	46.10		77.15	67.07	52.91	65.60
 DeepSeek V3.2	61.52	54.59	43.71	53.57		73.65	70.20	62.60	68.90
 Claude-4.6-Sonnet	67.58	62.01	60.12	63.20		73.98	67.78	62.32	68.04

A Additional Results

Table 5 presents the performance distribution across various repository categories, while Table 6 provides the aggregate success rate (SR) averaged across all test cases. For clarity, we categorize the evaluation set into five functional domains: **Form** (Serialization & Data Formats), **Crypto** (Cryptography & Encoding), **Struc** (Data Structures & Utilities), **Math/Sci** (Mathematics & Science), and **Spec** (Specialized Tools).

A primary limitation of this study is the constrained evaluation of the Mini-SWE-Agent compared to the OpenHands-bash framework. Specifically, proprietary models such as GPT and Gemini were excluded from the Mini-SWE-Agent suite due to computational resource constraints and associated inference costs. We aim to expand the model coverage and provide a more comprehensive benchmark in future iterations of this work.

Dataset Details Initially, we prompt the language model to generate 60 test cases per file. After filtering, we save the successful test cases and successful test files. The remaining test files (samples that pass the filtering stage) have (about) 40 cases on average, while the minimum and maximum number of test cases per file are 20 and 60.

Table 7: Evaluation on  **Py2JS**  with DeepSeek V3.1 as the backbone model. We present the averaged success rate across all test cases across Easy, Medium, and Hard task difficulties, and the maximum retry times are set to 0 (coding only), 1 (coding-testing-refining), and 2 (coding-testing-refining-testing-refining).

Method	 Openhands-bash				 Mini-SWE-Agent			
	Easy	Medium	Hard	Avg.	Easy	Medium	Hard	Avg.
 Retry-0	55.93	45.51	36.79	46.10	77.15	67.07	52.19	65.60
 Retry-1	65.85	63.74	54.31	61.43	81.21	73.72	61.70	72.32
 Retry-2	67.50	58.57	59.69	61.64	78.56	72.66	67.53	72.93

B A Deeper Look into ACE

Table 7 reports the success rates achieved through the ACE workflow. A comparative analysis of the *Retry-1* and *Retry-2* iterations reveals that while the individual test case success rate exhibits only a marginal increase, the overall sample-level pass rate improves significantly (see Table 4). This divergence suggests that the ACE framework is particularly effective at resolving "corner case" failures within samples; by rectifying these pivotal edge cases, the framework disproportionately enhances the aggregate pass rate of entire repositories.

Building upon the empirical results of ACE, we propose two strategic directions for future research:

- **Comprehensive Test Suite Synthesis:** For agent-based software development, it is imperative to construct exhaustive test suites. This enables coding agents to leverage the ACE framework for rigorous self-verification, ensuring that generated codebases adhere to complex functional requirements.
- **High-Fidelity Predictive Oracles:** In scenarios where large-scale test cases are difficult to procure, developing advanced "oracle models" is essential. Such models must possess the capacity to precisely predict execution outputs given specific inputs and source code. Integrating these specialized testing models into the ACE workflow could substantially augment the reasoning and self-correction capabilities of coding agents.

C Prompts

We provide the prompt templates used in our agentic evaluation and ACE loop. Placeholders enclosed by angle brackets are instantiated for each task with the corresponding source file, target path, and repository-specific dependency constraints.

System Prompt

```
[system]
You are an AI expert with Linux terminal access. Use your reasoning ability before calling any tools.

[tool]
execute_shell(command): Execute a Linux shell command and return its output.
```

Py2JS Generation Prompt

```
[user]
You are an expert cross-language migration engineer (Python to Node.js). Read the following Python
source code:

--- Source (<DATASET_ROOT>/<RELATIVE_PATH>) ---
<PYTHON_SOURCE_CODE>
---
<WHITE_BOX_TEST_CASES>
Requirements:
1. Environment: Write pure JavaScript for Node.js using ES Modules (ESM). Use 'import'/'export'. Do
   NOT use 'require()' or 'module.exports'.
   - All generated library and entry files must use the '.mjs' suffix.
2. CLI arguments: The JS file must accept exactly the same command-line arguments as the Python
   file (same names, defaults, required fields). Parse via 'process.argv'. 'node test.mjs --arg val'
   must behave identically to 'python test.py --arg val'.
3. Logic and output: Algorithm logic, numeric precision, and string formatting must exactly match
   the Python source. 'console.log' output must be byte-for-byte identical to Python 'print' output.
4. Zero external dependencies: Do NOT use any npm packages (e.g., yargs, argparse). Only import
   local files. Use only Node.js built-in modules (e.g., 'node:fs', 'node:path', 'node:url').
5. Project structure: Generate library files in '<OUTPUT_BASE>/packages/<RELATIVE_PATH_WITHOUT_PY>
   _pkg'. Split library code into modules with 'export'. In ESM, 'import' statements must include
   full file extensions (e.g., 'import { x } from './utils.mjs'). Working directory: '<OUTPUT_BASE>
   >'.
6. Black-box implementation: Do not read Python library source. Infer behavior from the interface
   and re-implement in JS. Do NOT reference '<FORBIDDEN_JS_APIS_OR_LIBRARIES>'.

Output library files first (.mjs), then the entry test file '<OUTPUT_MJS_PATH>'.
```


C2Rust Generation Prompt

```
[user]
You are an expert C++ to Rust migration engineer. Read the following C++ source code:

--- Source (<CPP_SOURCE_PATH>) ---
<CPP_SOURCE_CODE>
---
<WHITE_BOX_TEST_CASES>
Requirements:
1. Environment: Write pure Rust using the 2021 edition. Code must compile with 'rustc' or as a
    Cargo project.
2. CLI arguments: The Rust binary must accept exactly the same command-line arguments as the C++
    binary (same names, defaults, and required fields). Parse args via 'std::env::args()'.
3. Logic and output: Algorithm logic, numeric precision, and string formatting must exactly match
    the C++ source. 'println!' output must be byte-for-byte identical to 'std::cout' output.
4. Zero external dependencies: Do NOT use any crates from crates.io (only 'std'). Forbidden crates
    for this repo: <FORBIDDEN_CRATES>. Implement all functionality from scratch using the standard
    library.
5. Project structure: Create a complete Cargo project in '<PACKAGE_DIR>'. Organize library code
    into modules. Place the entry test file '<RUST_FILENAME>' in the package root '<PACKAGE_DIR>'.
6. Black-box implementation: Do not read the C++ library source. Infer behavior from the interface,
    then re-implement using 'std'.
7. Output:
    - Create and implement library files inside '<PACKAGE_DIR>'.
    - Save the entry file to '<OUTPUT_RUST_PATH>'.
    - Compile and produce an executable at '<OUTPUT_BINARY_PATH>'.

Hint: '<COMPILED_CPP_BINARY>' is the compiled C++ binary -- use it to debug with any arguments.
```

ACE Test Generation Prompt

```
[user]
Read the code and save test parameters to <TEST_CASES_JSON_PATH> as a JSON array:

You are a test engineer. Read the following Python code and generate 5 different sets of command-line
arguments to test its logic.
Cover: normal input, boundary values, and possible error inputs.

Source code:
<PYTHON_SOURCE_CODE>

Output a JSON array of strings only, e.g.: ["--input 1", "--input 10 --verbose", ""]
No explanations, no output values -- only input argument strings.
```

ACE Refinement Prompt Suffix

```
[user]
<PY2JS_GENERATION_PROMPT>

[IMPORTANT] Do not rewrite from scratch -- fix the existing code.
Current path: <OUTPUT_MJS_PATH>
Library path: <OUTPUT_BASE>/packages/<RELATIVE_PATH_WITHOUT_PY>_pkg
Previous failure:
<FAILURE_MESSAGE>
```

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.